Forex Trading Strategy Analysis

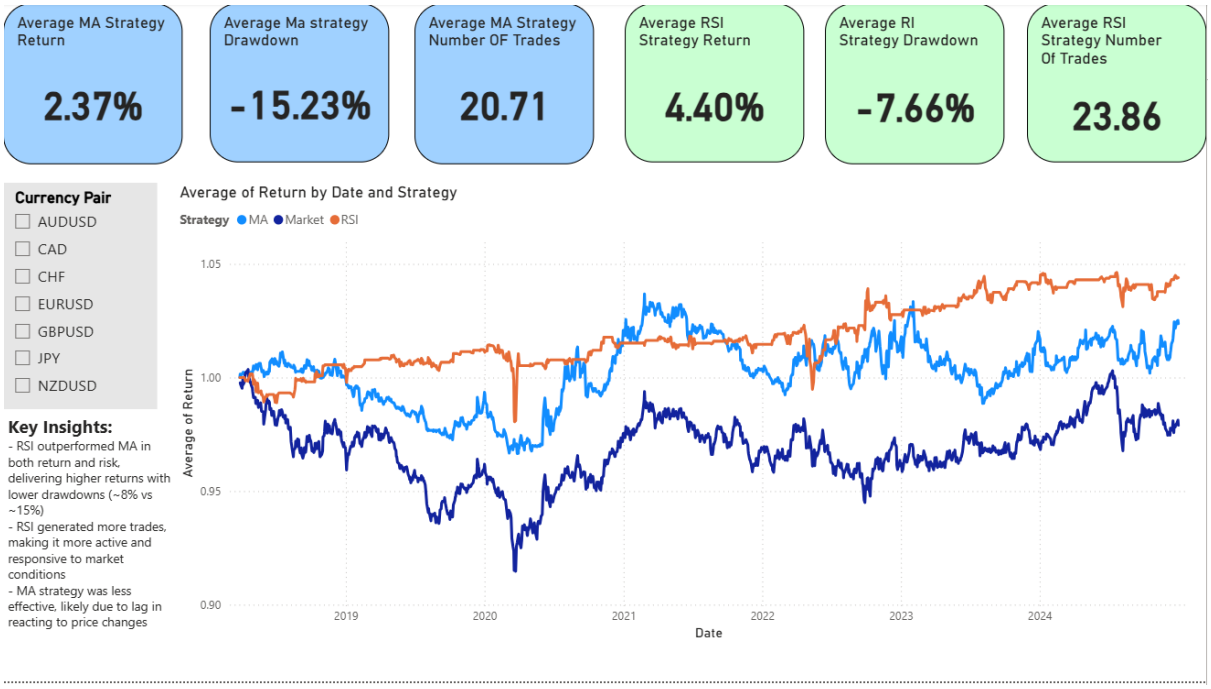
Project Overview

This project was undertaken to explore whether simple technical indicators could be used to create profitable systematic trading strategies in the foreign exchange market. Rather than manually reviewing charts, I wanted to automate the process of generating trading signals, backtesting strategies across multiple currency pairs and analysing their performance using Python and Power BI.

The project consisted of two independent strategies:

- Moving Average (MA) crossover strategy
- Relative Strength Index (RSI) strategy

Historical market data was downloaded using Python before each strategy was backtested across several major FX pairs. The resulting trade history and performance metrics were exported into Power BI to allow interactive comparison between strategies.



Methodology

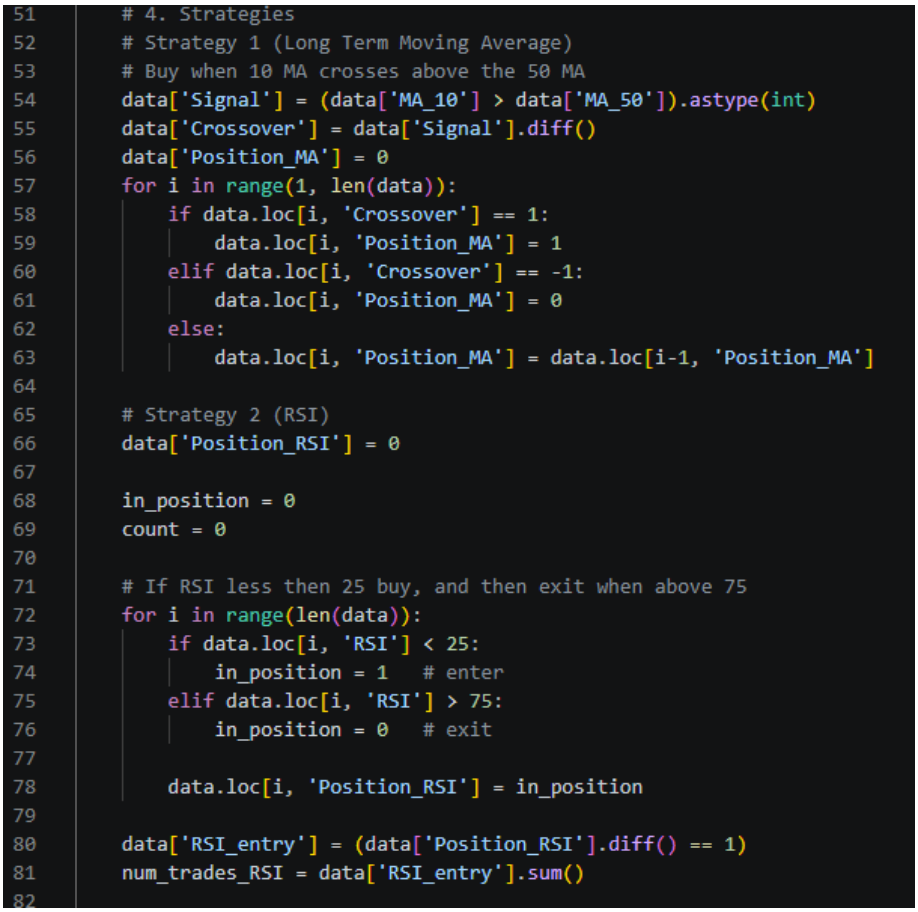
The workflow consisted of four stages:

1. Download historical FX data using Python (yfinance)
2. Generate buy and sell signals using MA and RSI indicators

3. Backtest each strategy and calculate key performance metrics
4. Visualise the results in Power BI

Key metrics analysed included:

- Total Return
- Maximum Drawdown
- Number of Trades
- Cumulative Return
- Strategy Comparison



```
# 4. Strategies
# Strategy 1 (Long Term Moving Average)
# Buy when 10 MA crosses above the 50 MA
data['Signal'] = (data['MA_10'] > data['MA_50']).astype(int)
data['Crossover'] = data['Signal'].diff()
data['Position_MA'] = 0
for i in range(1, len(data)):
    if data.loc[i, 'Crossover'] == 1:
        data.loc[i, 'Position_MA'] = 1
    elif data.loc[i, 'Crossover'] == -1:
        data.loc[i, 'Position_MA'] = 0
    else:
        data.loc[i, 'Position_MA'] = data.loc[i-1, 'Position_MA']

# Strategy 2 (RSI)
data['Position_RSI'] = 0

in_position = 0
count = 0

# If RSI less then 25 buy, and then exit when above 75
for i in range(len(data)):
    if data.loc[i, 'RSI'] < 25:
        in_position = 1   # enter
    elif data.loc[i, 'RSI'] > 75:
        in_position = 0   # exit

    data.loc[i, 'Position_RSI'] = in_position

data['RSI_entry'] = (data['Position_RSI'].diff() == 1)
num_trades_RSI = data['RSI_entry'].sum()
```

Parameter Testing

To explore how indicator settings affected performance, I created a separate Python script that allowed me to test different Moving Average periods and RSI thresholds across the same set of currency pairs. This helped compare how changes in parameters influenced returns, drawdown and trading activity, while reinforcing the importance of selecting appropriate strategy settings.

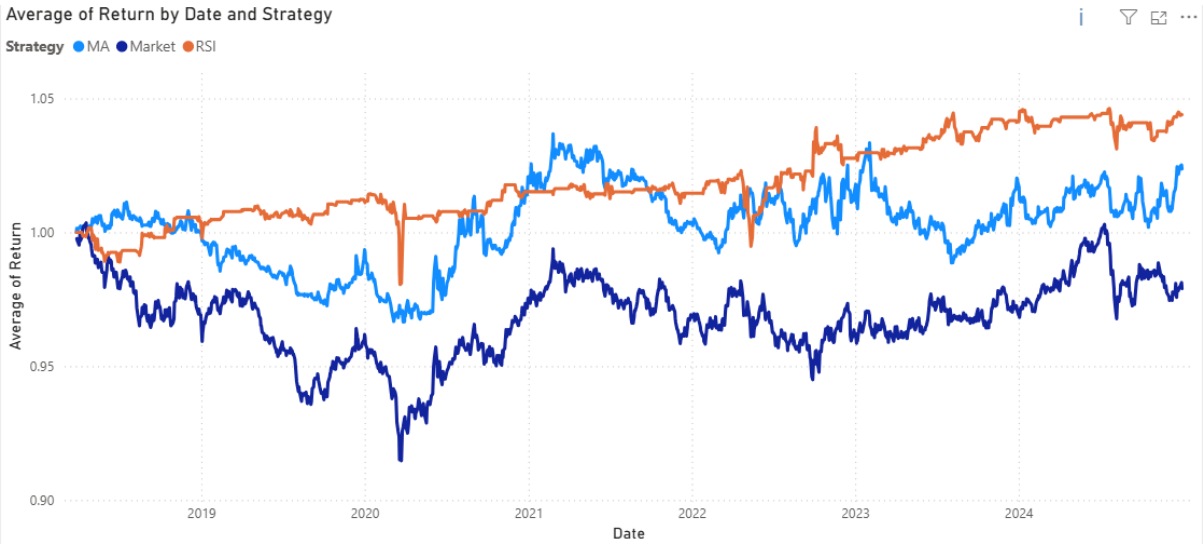
Results

Across the tested currency pairs, the RSI strategy consistently outperformed the Moving Average strategy.

Metric	MA Strategy	RSI Strategy
Average Return	**2.37%**	**4.40%**
Average Drawdown	**-15.23%**	**-7.66%**
Average Trades	**20.7**	**23.9**

The RSI strategy generated both higher returns and significantly lower drawdowns, while also producing slightly more trading opportunities.

However, neither strategy consistently outperformed the market benchmark over the full testing period.



Discussion

One of the main outcomes of this project was recognising the limitations of relying on individual technical indicators.

Although both strategies produced periods of profitable performance, they also experienced prolonged underperformance. This demonstrated that simple indicator-based systems are often insufficient when used in isolation. Market conditions such as volatility, trend strength, macroeconomic events and transaction costs all influence trading performance and were not fully incorporated into these models.

This reinforced the importance of combining multiple sources of information rather than expecting a single indicator to consistently generate profitable trading signals.

Skills Demonstrated

- Python
- Pandas
- Data cleaning
- Algorithmic backtesting
- Financial data analysis
- Excel
- Power BI
- Data visualisation
- Performance evaluation